

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: 45 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad Técnica	Se evidencia al asegurar que los punteros de la lista circular siempre cierren el ciclo para evitar desbordamientos de memoria.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Fortalecer la capacidad de abstracción para el diseño y construcción de estructuras de datos dinámicas eficientes.
Competencia Específica	El estudiante desarrolla la habilidad de implementar y diferenciar el comportamiento de punteros en listas doblemente enlazadas y listas circulares.

2. Palabras Clave

NodoDoble: Clase que contiene un valor entero y dos referencias (Siguiente y Anterior) para la navegación bidireccional.

Lista Circular: Variante de lista donde el último nodo apunta al primero, permitiendo un recorrido infinito.

Graphviz (DOT): Herramienta de software para la visualización de estructuras mediante código basado en grafos.

3. Resumen

Este ejercicio presenta el desarrollo guiado de dos estructuras de datos fundamentales: la Lista Doblemente Enlazada y la Lista Circular Doble. El propósito es mostrar al estudiante cómo gestionar las referencias Anterior y Siguiente para realizar operaciones de inserción, eliminación y visualización gráfica mediante archivos .dot y procesos externos de Graphviz.

4.Contenido

Descripción del ejercicio

El ejercicio consiste en la implementación de las clases necesarias para gestionar listas dinámicas en C#, incluyendo la generación automática de diagramas representativos de la estructura actual de los datos en memoria.

Objetivo de la demostración

Demostrar el procedimiento para manipular punteros en listas circulares, utilizando C# y Graphviz, con el fin de que el estudiante comprenda la lógica de cierre de ciclos y la navegación bidireccional.

Recursos necesarios

- **Software:** Visual Studio o VS Code con el SDK de .NET.
- **Herramientas:** Graphviz instalado y configurado en las variables de entorno (comando dot).
- **Archivos:** nodoDoble.cs, listaDoble.cs, listaCircular.cs y Program.cs

Desarrollo paso a paso

- **Paso 1: Definición del Nodo.** Crear la clase `NodoDoble` con propiedades auto-implementadas para `Valor`, `Siguiente` y `Anterior`. Esto establece la unidad básica de almacenamiento.
- **Paso 2: Inserción y Enlace.** Implementar `AgregarAlFinal` en `ListaCircular`. Es crucial observar cómo, tras agregar un nuevo nodo, `ultimo.Siguiente` debe apuntar a `primero` y `primero.Anterior` a `ultimo`.
- **Paso 3: Lógica de Eliminación.** Desarrollar el método `Eliminar`. El estudiante debe verificar que, al borrar el primer elemento, los enlaces del último nodo se actualicen para mantener la integridad del círculo.
- **Paso 4: Visualización.** Ejecutar el método `Graficar`, el cual utiliza `StringBuilder` para crear un archivo de texto con lenguaje DOT y posteriormente invoca el proceso `dot.exe` para generar la imagen PNG.

Observaciones importantes

- **Error común:** No actualizar el puntero `Anterior` al eliminar un nodo en medio de la lista, lo que rompe la navegación hacia atrás.
- **Buena práctica:** Usar `Math.Abs(actual.GetHashCode())` para generar identificadores únicos para cada nodo en el archivo de Graphviz.

Tabla 1: Comparativa de Métodos

Elemento	Lista Doble	Lista Circular
Fin de lista	El último nodo apunta a null.	El último nodo apunta al primero.
Inicio de lista	El primer nodo apunta a null en su anterior.	El primer nodo apunta al último en su anterior.
Recorrido	Termina cuando llega a un nulo.	Requiere una bandera o condición para no entrar en bucle infinito.

Resultado esperado

Al ejecutar el programa, se deben generar dos archivos de imagen (lista_doble.png y lista_circular.png) que muestren los nodos conectados con flechas azules para el siguiente y flechas rojas punteadas para el anterior.

Aplicación práctica

Este ejercicio se relaciona directamente con el desarrollo de software y la ingeniería de sistemas en los siguientes aspectos:

- **Gestión de Memoria en Sistemas Operativos:** Las listas circulares se utilizan comúnmente en la planificación de procesos (Round Robin), donde el sistema operativo debe recorrer una lista de tareas pendientes de forma cíclica para asignar tiempo de CPU a cada una.
- **Desarrollo de Interfaces de Usuario:** La lógica de la lista doblemente enlazada es la base para implementar funciones de "Deshacer" (Undo) y "Rehacer" (Redo) en editores de texto o software de diseño, permitiendo la navegación bidireccional entre estados.
- **Sistemas de Reproducción Multimedia:** Las listas de reproducción (playlists) que permiten la opción de "repetir todo" funcionan internamente como listas circulares, donde al terminar la última canción, el puntero "Siguiente" redirige automáticamente al primer elemento.
- **Continuidad en el Laboratorio:** Este ejercicio sirve como base técnica para la siguiente unidad de "Listas Ortogonales" y "Matrices Dispersas", donde la comprensión profunda de los punteros **Anterior** y **Siguiente** es indispensable para manejar estructuras multidimensionales.
- **Visualización de Datos:** El uso de herramientas como Graphviz enseña al estudiante la importancia de la observabilidad en el desarrollo, permitiendo validar visualmente que la lógica de punteros implementada en el código coincide con el diseño teórico esperado.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- **Dificultades:** El manejo de procesos externos (Process.Start) suele fallar si el estudiante no tiene Graphviz en el PATH del sistema.
- **Recomendación:** Enfatizar la depuración paso a paso para que vean cómo cambian las referencias de memoria antes de intentar graficar.

6. Referencias

1. Microsoft Learn, Documentación de C# y .NET, 2026.
2. Graphviz - Graph Visualization Software, Documentation (<https://graphviz.org/>).